

Category: Developer Productivity / API Cost Analytics

Target users: API-key agent users.

Repository: github.com/LiuZongrun7/Mobile_Group_20

1. App idea and user problem

Students and independent developers may use several coding agents, but their logs and provider views are separate. Token totals alone do not explain expenses. They need a shared view of recorded spending and an accessible way to ask what changed or whether a budget is at risk.

TokenTrail is an Android API-cost app for users of **Codex with OpenAI**, **ZCode with Xiaomi MiMo** and **DeepSeek Harness (DSH) with DeepSeek**. Its analytics home screen shows estimated costs, usage, sessions and budgets from available, authorised records of pay-as-you-go API calls. A **forum** provides model news and tips, and a **monthly tower-defence game** reflects already logged usage as progress; both are supporting extensions.

An **interactive home-screen AI assistant** provides an “**Ask AI / hold to talk**” entry above navigation. Tap to open an expandable question panel, or hold to dictate a question. The assistant explains cost changes, signals budget warnings and compares logged agent costs with evidence. It belongs to **TokenTrail's analytics experience**, works independently of the game, and **does not run, route or control** coding agents.

Scope is **pay-as-you-go API usage**, estimated from dated official-rate snapshots. **Token Plan and other subscriptions are excluded**, including their usage and quotas: subscription consumption cannot be treated as a per-call API charge. An API key alone cannot recover an agent's history. Missing logs/rates remain visible; amounts are estimates, and the assistant's own model usage and cost are recorded separately.

2. Related products and opportunity

The review compares first-party API billing views with existing cross-model monitoring tools on data scope, cost provenance and decision support.

Product	Relevant capability	Gap addressed by TokenTrail
Codex	API-key coding agent billed through OpenAI Platform.	Dated estimates from available Codex logs.
ZCode	Supports Xiaomi MiMo as a model provider.	Import usable MiMo API logs; exclude Token Plan usage.
DSH / dsh-context	DSH runs coding sessions; its context plugin visualises tokens, cache and activity.	Import usable DSH logs into Android; no live agent control.
LiteLLM	Gateway spend tracking and budgets for proxied calls.	Imports records without proxying agent traffic.
dsh-pet	Floating companion driven by DSH session events.	Interactive home-screen assistant with questions and evidence-linked advice.

3. Proposed contribution

Unified analytics. De-duplicate authorised logs and show source, coverage and session detail only where records support it. **Reproducible costs.** Java applies dated official rates by provider, model, token class and service tier, preserving the price version. **Evidence-based AI advice.** Text or speech questions obtain calculated evidence from read-only tools; replies disclose sample size and missing data. Compare recorded costs, tokens and cache use, not answer quality; no task ratings are required. Values remain *Estimated*, *Partial estimate* or *Unavailable*.

Scope rule: API-cost analytics and the home-screen assistant are core; forum and game are supporting extensions. Live cross-agent status and system-wide overlays are outside the first release.

4. Planned functionality and the four data aspects

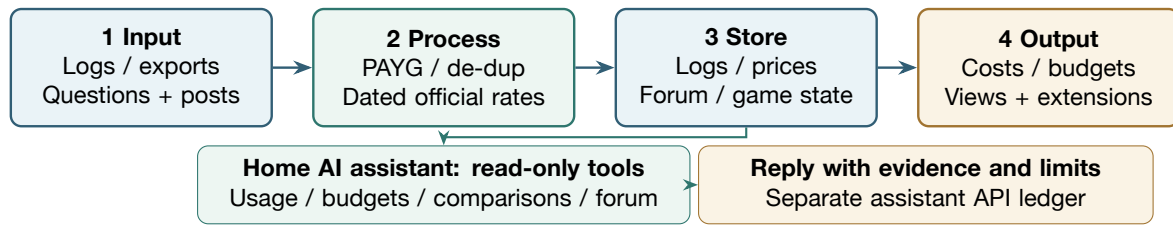
Aspect	Planned implementation	User value / evidence
Data input	Profiles, billing mode, budgets; authorised logs, text/confirmed speech, posts and game actions.	Validate sources and inputs; exclude plan usage; retain text fallback.
Data processing	Filter/de-duplicate logs; price usage; read-only summaries; rank posts and convert game resources.	Traceable costs/advice; no duplicate game rewards; labelled community context.
Data storage	Room: local records. Server-side store: user sync, budgets, game state and assistant ledger; public posts separate.	Account-scoped access isolates accounts; game/assistant records remain separate.
Data output	Costs, heatmap, sessions, budgets, AI evidence; forum lists and real-time battle feedback.	Explicit estimate/missing-data states; assistant API costs shown separately.

5. Core features and implementation approach

The analytics dashboard and assistant form the main experience; Forum and Game are supporting tabs.

- **Account and profiles.** Register, log in/out and reset passwords. Record the coding tool, provider, billing mode and a non-secret key label; coding-agent credentials are not required.
- **Usage, costs and sessions.** Import and de-duplicate authorised records. Only confirmed pay-as-you-go coding-agent records enter totals, budgets and game resources; exclude Token Plan, subscriptions and unknown billing modes. Match timestamps, provider/model, token classes and pricing tiers to official snapshots, including cache and time-based rates where applicable. Session attribution requires identifying logs; aggregates never become invented sessions. Source-billed amounts are separate reconciliation evidence, not replacements for estimates.
- **Budget and price updates.** Show estimated spend, coverage, budget progress and advisory alerts; forecast only from comparable history. Preserve source URL, currency, effective time and price version; CNY conversion retains its exchange-rate snapshot. Missing historical rates or insufficient timing detail mean unavailable cost, not zero or a backdated current price. No payments or external hard caps.
- **Interactive AI assistant and separate API cost tracking.** A floating “Ask AI / hold to talk” entry opens the home-screen panel. Type, or dictate and review/edit the transcript before sending. Request microphone permission on use; denial or failure leaves typing available. Raw audio is not retained by default. A Java service calls one selected OpenAI, Xiaomi MiMo or DeepSeek pay-as-you-go model and reads validated summaries. Replies explain spending/cache changes and compare costs only when comparable logs support them, citing coverage, price versions and missing data. Community tips are supplementary, never pricing authority. Rule-based alerts need no model call; assistant tokens/costs have a separate ledger, excluded from coding-agent totals and game resources.
- **Supporting forum.** Show linked official news, user posts and replies. Rank by recency/helpful votes with deterministic ties. Label community tips and keep public content separate from private usage.
- **Supporting tower-defence game.** A separate Game tab offers monthly progression from already logged pay-as-you-go coding-agent tokens. Input, cache and output map to resource pools for towers and walls. Players place/upgrade defences, protect a core and start real-time waves; health, wave progress and battle results provide feedback. Save monthly game state separately; playing never changes usage, cost or budget totals. No extra API spending is needed, and re-importing logs grants no duplicate resources.

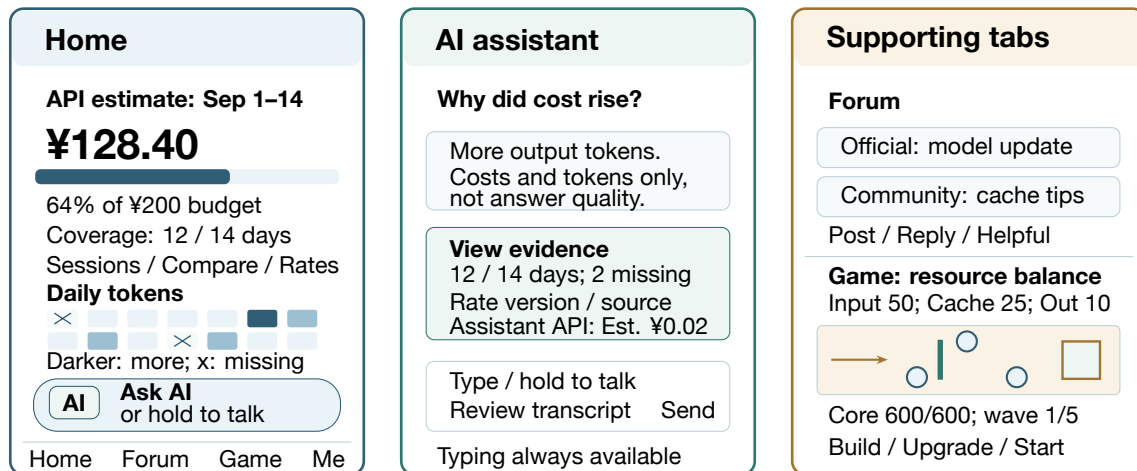
6. Data flow and limitations



PAYG means pay-as-you-go. Verify log version and billing mode; DSH may require JSONL or Zstandard decoding. Five read-only tools—*getUsageSummary*, *getBudgetStatus*, *compareAgentCosts*, *getForumHighlights* and *getMyThreads*—enforce the caller’s UID and bounded queries. Models receive summaries, not secrets or raw agent prompts; missing logs mean no personalised cost claim. No coding-agent control.

7. Proposed Android interface

Home leads with analytics and the assistant entry. Tap to ask; hold to dictate, then confirm the transcript. Sessions expose source, date and token classes only where logs support them; comparisons use matching filters. Forum and Game are supporting tabs. All sketches use illustrative September 2026 data; assistant API costs are excluded from the home total.



Interaction and accessibility. Use labelled touch targets, text scaling and rotation support. Missing logs/rates show *Unavailable*, not zero. Loading, error, listening and permission-denied states are explicit; typing and analytics remain available if the model fails. The assistant never covers data or navigation.

8. Technology and responsible reuse

Layer	Planned tools and rationale
Android interface	Java/XML, Material Components, Navigation and ViewModel/LiveData; RecyclerView lists, speech-to-text and typed fallback.
Network and work	Retrofit/OkHttp connects the Java advice service; bounded read-only tools and one model API. WorkManager checks prices.
Persistence and security	Room stores usage, prices and history; the team’s own server supports sync and public posts. The service authenticates every request and scopes every read to the caller’s account; no client holds a model key.
Open-source reuse	Room (Apache-2.0): used for local records. MPAndroidChart (Apache-2.0): added dependency; charts planned. Room with a View (Apache-2.0): archived architecture reference, no copied code. dsh-context/LiteLLM: comparison only; dsh-pet: entry inspiration only. Game engine: original. Record reused files, versions, licences and art sources.

9. Work plan through the final submission

Checkpoints are approximately Alpha (weeks 8–9), Beta (11–12) and Final (15–16); implementation and documentation are planned through week 15, with regular GitHub commits.

Course weeks	Work	Checkpoint / acceptance
1–3	Verify Codex/ZCode/DSH samples, billing modes, rate dimensions and UI; define access rules.	Verified sources; isolated accounts.
4–7	Build import, storage, dated pricing and analytics; define the five read-only tool contracts.	Correct rates; idempotent import.
8–9: Alpha	Login → profile → import → cost → home.	Separate real/sample data; source, GitHub link and MP4.
10–12: Beta	Wang: assistant and forum using Zhang's queries. Liu: settlement and real-time game.	Cited evidence; typing fallback; ranked posts; smooth waves.
13–15: Final	Validate advice, cost separation, game balance, accessibility and isolation; complete documentation.	Demo, tests, report/video; licence records.

10. Team work and evidence of completion

Zhang Li (24107757): authentication, log import/storage, dated pricing, budgets, dashboard and data-query interfaces.

Wang Tingdong (24107759): home-screen AI assistant, text/voice panel, Java advice service, five read-only tools and assistant API ledger; forum news, posts/replies and ranking.

Liu Zongrun (24107745): tower-defence engine/UI, resource settlement, monthly state, placement/upgrades and enemy waves.

All share UI/UX, accessibility and integration, and provide commits/module acceptance evidence.

Demo: Import logs → cost/session/budget views → ask “Why did cost rise?” by text/confirmed speech → evidence and separate assistant API cost. Then show forum posts and a player-started game wave.

Validation: Test plan exclusion, log formats, token classes, dated/time-based rates, missing data, de-duplication and account/tool isolation. Check replies against evidence, text fallback and ledger separation; game re-imports grant no duplicate resources. Sample data never enters real totals.

11. Functional requirements and improvements

Existing feature	TokenTrail improvement / addition	Type
Cost dashboards	Cross-provider dated estimates with source and coverage.	Improved
Session viewers	Log-supported sessions and token-class breakdowns.	Improved
Billing budgets	Cross-provider estimated budget and advisory alerts.	Improved
AI assistants	Home AI: text/voice, usage evidence and separate API ledger.	New
Forums	Linked official news, posts/replies and context for advice.	Improved
Tower defence	Monthly game resources from verified coding-agent API logs.	New

12. Sources and project references

Pricing: OpenAI; Xiaomi MiMo; DeepSeek. Store dated snapshots; these links are not historical rates.

Agents/data: Codex auth; ZCode configuration; DSH providers; DSH session logs.

Implementation: Function calling; Java SDK; Android security best practices. Project/source links in §2 and §8; MPAndroidChart licence.